Misión: Programa espacial en la NASA

Manuela Planelles Lucas.

1° DAW - Entornos de Desarrollo.

11.03.2026

Contenido

Parte I..... 3
Parte II..... 4
Parte III..... 5
Parte IV..... 8
Screenshot..... 9

En esta etapa de la expedición, se han realizado maniobras de pruebas unitarias utilizando JUnit 5 dentro del centro de mando IntelliJ IDEA. La misión consistía en verificar la integridad de la clase Alien, encargada de custodiar los datos de especies biológicas externas, asegurando que sus protocolos de respuesta sean estables antes de autorizar su despliegue en un entorno de operaciones real.

Este informe de misión se estructura en cuatro fases: la primera documenta la preparación del entorno y la configuración del sistema de pruebas, la segunda recoge la implementación de los casos de prueba sobre la clase Alien, la tercera analiza los resultados obtenidos, incluyendo el comportamiento ante fallos y la cobertura del código verificado, y la cuarta recoge las conclusiones obtenidas tras el proceso completo.

Parte I.

¿En qué carpeta se guarda el código fuente de la aplicación?

El código fuente de la aplicación se guarda en la carpeta src/main/java. Esta carpeta contiene todas las clases que forman el programa en sí, es decir, el código que se ejecutará en producción. En este proyecto, es donde se ha colocado la clase Alien.java, ya que es la clase principal sobre la que trabajamos.

¿En qué carpeta se guardan las pruebas unitarias?

Las pruebas unitarias se guardan en la carpeta src/test/java. Esta carpeta está diseñada específicamente para alojar las clases de test, separándolas del código funcional de la aplicación, y es donde se ha creado la clase AlienTest.java.

Maven la establece por defecto para que el código de pruebas no se incluya en el programa final cuando se compila para producción, manteniendo así el proyecto ordenado y limpio.

[Ver Figura 1]

¿Cómo se añade JUnit 5 al proyecto y qué es la anotación @Test?

Se ha optado por consultar los manuales de la asignatura de Programación, siguiendo el proceso de configuración automática que facilita IntelliJ.

El primer paso ha consistido en verificar que el proyecto cuenta con la estructura adecuada, confirmando que la carpeta test/java aparece correctamente identificada por el entorno.

Para la creación de las pruebas, se ha utilizado la herramienta asistida del IDE: al situar el cursor sobre la clase Alien, se ha activado la opción Create test. En este punto, se ha seleccionado la versión de JUnit y los métodos específicos que se querían supervisar, permitiendo que el propio entorno genere automáticamente el archivo AlienTest.java. Finalmente, se han incluido las anotaciones @Test en cada método generado, asegurando así que JUnit los reconozca y los ejecute como casos de prueba válidos.

Parte II.

En esta fase se han implementado un total de 8 tests sobre la clase Alien.

A continuación, se describe qué comprueba cada uno y qué assertion de JUnit se ha utilizado en cada maniobra de verificación:

`testConstructorStoresName()` — Comprueba que el constructor con parámetros almacena correctamente el nombre del alien. Se utiliza assertEquals para verificar que getName() devuelve el valor esperado.

`testConstructorStoresPlanetId()` — Comprueba que el constructor almacena correctamente el identificador del planeta. Se utiliza assertEquals para verificar que getPlanetId() devuelve el valor esperado.

`testToString()` — Verifica que el método toString() devuelve la cadena en el formato exacto requerido: "Zorg MARS01". Se utiliza assertEquals para asegurar la precisión del texto.

`testAliensWithSamePlanetIdAreEqual()` — Confirma que dos objetos Alien con el mismo planetId se consideran iguales según el método equals(). Se utiliza assertTrue para validar que la condición es verdadera.

`testAliensWithDifferentPlanetIdAreNotEqual()` — Verifica que dos aliens con distinto planetId no se identifiquen como iguales. Se utiliza assertFalse para confirmar que la comparación es negativa.

`testConstructorNotNull()` — Asegura que el objeto creado mediante el constructor no sea nulo, validando que la instanciación se ha realizado con éxito. Se utiliza assertNotNull.

`testSetName()` — Comprueba que setName() actualiza el nombre del registro correctamente. Se modifica el valor y se verifica el cambio con getName() mediante assertEquals.

`testSetPlanetId()` — Verifica que setPlanetId() actualiza el identificador del planeta sin errores. Se cambia el valor y se valida con getPlanetId() usando assertEquals.

[Ver Figura 2]

Parte III.

¿Qué significa que un test aparezca en verde?

Indica que la prueba se ha superado con éxito: el resultado coincide exactamente con lo que se espera. Es la señal de que el código se comporta de forma correcta.

¿Qué significa que un test aparezca en rojo?

Representa un fallo en la prueba: el resultado real no encaja con el esperado o se ha producido una excepción. Funciona como una alerta de que algo en el código bajo examen requiere una corrección.

¿Cómo se puede ejecutar un solo test?

En el entorno IntelliJ, basta con hacer clic derecho directamente sobre el método que lleva la anotación @Test y seleccionar la opción Run 'nombreDelTest()' en el menú que aparece.

¿Cómo se ejecutan todos los tests del proyecto?

Se puede iniciar la revisión completa haciendo clic derecho sobre la clase AlienTest y eligiendo Run 'AlienTest'. También es posible utilizar el icono de reproducción verde que se encuentra en el margen izquierdo, justo al lado del nombre de la clase.

[Ver Figura 3]

Análisis de un test con error: Siguiendo la práctica, se ha diseñado a propósito un test que error.

El objetivo es observar cómo reacciona JUnit ante un fallo y analizar la información que ofrece el sistema cuando algo no cuadra. El código añadido:

```
@Test
void testFallaIntencionadamente() {
    Alien alien = new Alien("Zorg", "MARS01");
    assertEquals("Xenomorph", alien.getName());
}
```

¿Qué valor aparece como expected?

Aparece "Xenomorph", que es el dato que se ha introducido manualmente en el assertEquals como el resultado que se esperaba obtener.

¿Qué valor aparece como actual?

Se muestra "Zorg", que es el valor real que devuelve el método getName(). Es el nombre con el que se configuró el objeto al principio.

¿Cómo ayuda ese mensaje a localizar el error?

Esto permite ver al instante si el fallo está en el código de la clase, que no entrega el dato correcto, o si el propio test está mal planteado.

¿Qué habría que cambiar para que el test pasara correctamente?

Para que la señal vuelva a estar en verde, solo habría que corregir el valor expected de "Xenomorph" a "Zorg". Al coincidir el valor esperado con el nombre real que guarda el constructor, la prueba se daría por superada.

[Ver Figura 4]

Una vez analizado el comportamiento del sistema ante el fallo, y tal como se solicita en el proyecto para restablecer la integridad de las pruebas, se ha procedido a corregir el valor expected de "Xenomorph" a "Zorg".

Tras volver a ejecutar el proceso de verificación, el sistema ha confirmado que los 9 tests han sido superados con éxito. Este ajuste final permite concluir la fase de pruebas con todos los indicadores en verde, asegurando que la clase Alien cumple ahora con los estándares de funcionamiento exigidos antes de su despliegue definitivo.

[Ver Figura 5]

¿Qué es la cobertura de código?

La cobertura de código es una métrica que señala qué porcentaje de las líneas de programación se han activado realmente durante las pruebas. Cuanto más alto sea este valor, mayor es la tranquilidad de saber que el comportamiento del sistema se ha verificado a fondo. Se trata de una herramienta clave para descubrir rincones del código que han quedado olvidados y no han sido supervisados por ningún test.

¿Qué significan los colores que aparecen en el editor?

Verde: Indica que la línea ha sido ejecutada por al menos uno de los tests. Se considera una zona segura y bajo control.

Rojo: Significa que la línea no ha sido visitada por ninguna prueba. Es una señal de que esa parte del código aún no ha sido comprobada.

Amarillo: Refleja que la línea se ha ejecutado solo a medias; por ejemplo, cuando en una condición if solo se ha explorado uno de los caminos posibles.

¿Qué partes de tu clase Alien han sido ejecutadas por los tests y cuáles no?

La mayoría de la clase ha sido verificada correctamente, incluyendo los constructores con datos, los métodos de acceso, el sistema de igualdad y el método toString().

Sin embargo, los resultados muestran una cobertura del 77% en métodos y del 60% en líneas. Esto se debe a que el constructor vacío y algunas rutas secundarias del código no se han utilizado en los tests. Al no ser necesarios para probar el comportamiento principal, han quedado sin ejecutar, permitiendo identificar qué partes están protegidas y cuáles permanecen en reserva.

[Ver Figura 6]

Parte IV.

La misión ha concluido con éxito. A través de esta práctica, se ha confirmado que las pruebas unitarias son una pieza fundamental del desarrollo; sin ellas, los errores se acumulan y complican la estabilidad del sistema.

Gracias a JUnit 5 e IntelliJ, se ha verificado sistemáticamente que la clase Alien responde correctamente en todos los escenarios previstos. Con los tests validados y la cobertura analizada, el código está listo para su despliegue.

Como medida de respaldo, el código fuente completo se encuentra disponible en el siguiente repositorio de GitHub, garantizando el acceso a la información en caso de que los archivos adjuntos presenten algún inconveniente: [Enlace Repositorio]

Screenshot

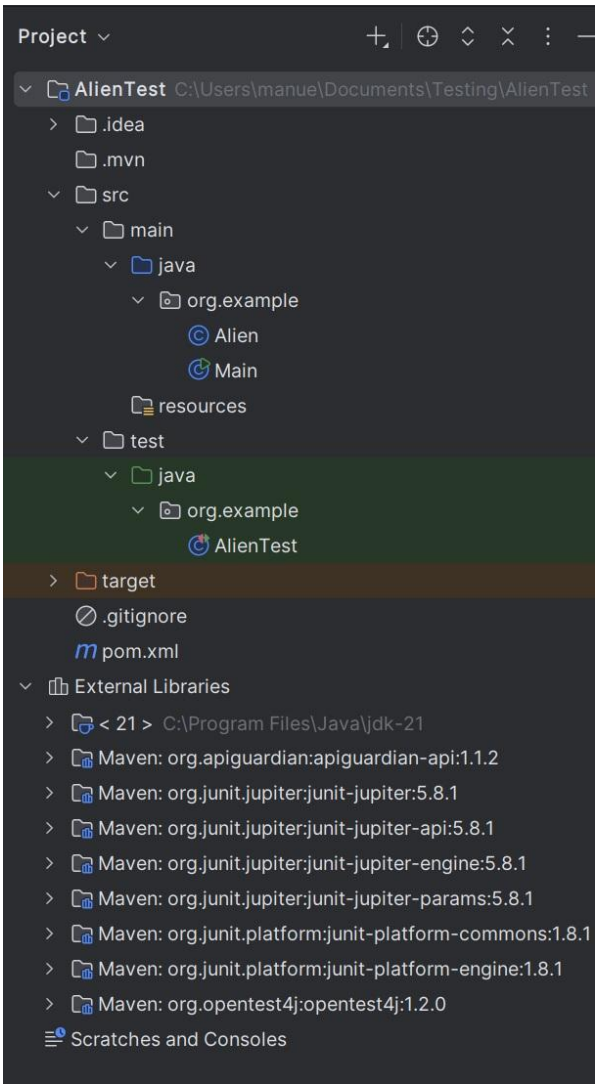

Figura 1 – Estructura del proyecto en IntelliJ. Se puede observar cómo el código fuente se encuentra en src/main/java y las pruebas unitarias en src/test/java, ambas bajo el paquete org.example.

pom.xml (AlienTest) | Main.java | Alien.java | AlienTest.java

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class AlienTest {

    @Test
    void testConstructorStoresName() {
        Alien a1 = new Alien( name: "Zorg",  planetId: "MARS01");
        assertEquals( expected: "Zorg", a1.getName());
    }

    @Test
    void testConstructorStoresPlanetId() {
        Alien a1 = new Alien( name: "Zorg",  planetId: "MARS01");
        assertEquals( expected: "MARS01", a1.getPlanetId());
    }

    @Test
    void testToString() {
        Alien a1 = new Alien( name: "Zorg",  planetId: "MARS01");
        assertEquals( expected: "Zorg MARS01", a1.toString());
    }

    @Test
    void testAliensWithSamePlanetIdAreEqual() {
        Alien a1 = new Alien( name: "Zorg",  planetId: "MARS01");
        Alien a3 = new Alien( name: "Zorg",  planetId: "MARS01");
        assertTrue(a1.equals(a3));
    }

    @Test
    void testAliensWithDifferentPlanetIdAreNotEqual() {
        Alien a1 = new Alien( name: "Zorg",  planetId: "MARS01");
        Alien a2 = new Alien( name: "Xenomorph",  planetId: "LV426");
        assertFalse(a1.equals(a2));
    }

    @Test
    void testConstructorNotNull() {
        Alien a1 = new Alien( name: "Zorg",  planetId: "MARS01");
        assertNotNull(a1);
    }

    @Test
    void testSetName() {
        Alien a1 = new Alien( name: "Zorg",  planetId: "MARS01");
        a1.setName("Xenomorph");
        assertEquals( expected: "Xenomorph", a1.getName());
    }

    @Test
    void testSetPlanetId() {
        Alien a1 = new Alien( name: "Zorg",  planetId: "MARS01");
        a1.setPlanetId("LV426");
        assertEquals( expected: "LV426", a1.getPlanetId());
    }
}
```

Figura 2 – Clase AlienTest.java con los 8 tests implementados. Se pueden observar las anotaciones @Test sobre cada método y las assertions utilizadas para verificar el comportamiento de la clase Alien.

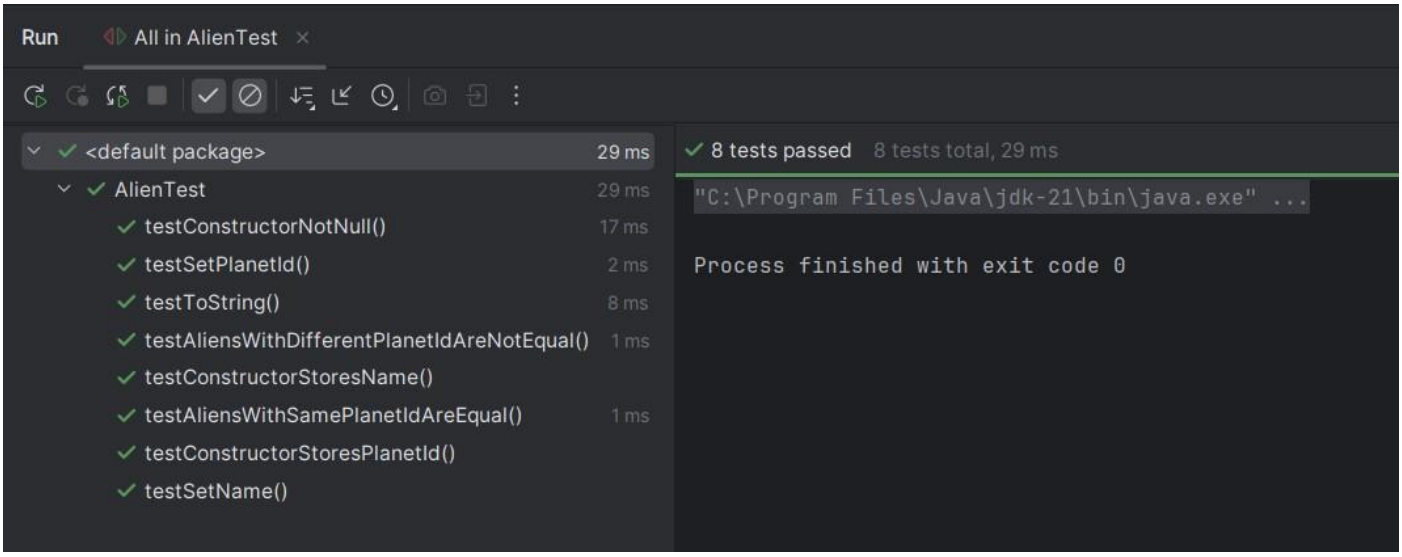

Figura 3 – Ejecución de los tests en verde. Se puede observar cómo los 8 tests implementados sobre la clase Alien han sido superados con éxito, mostrando el resultado 8 tests passed en el panel de ejecución de IntelliJ.

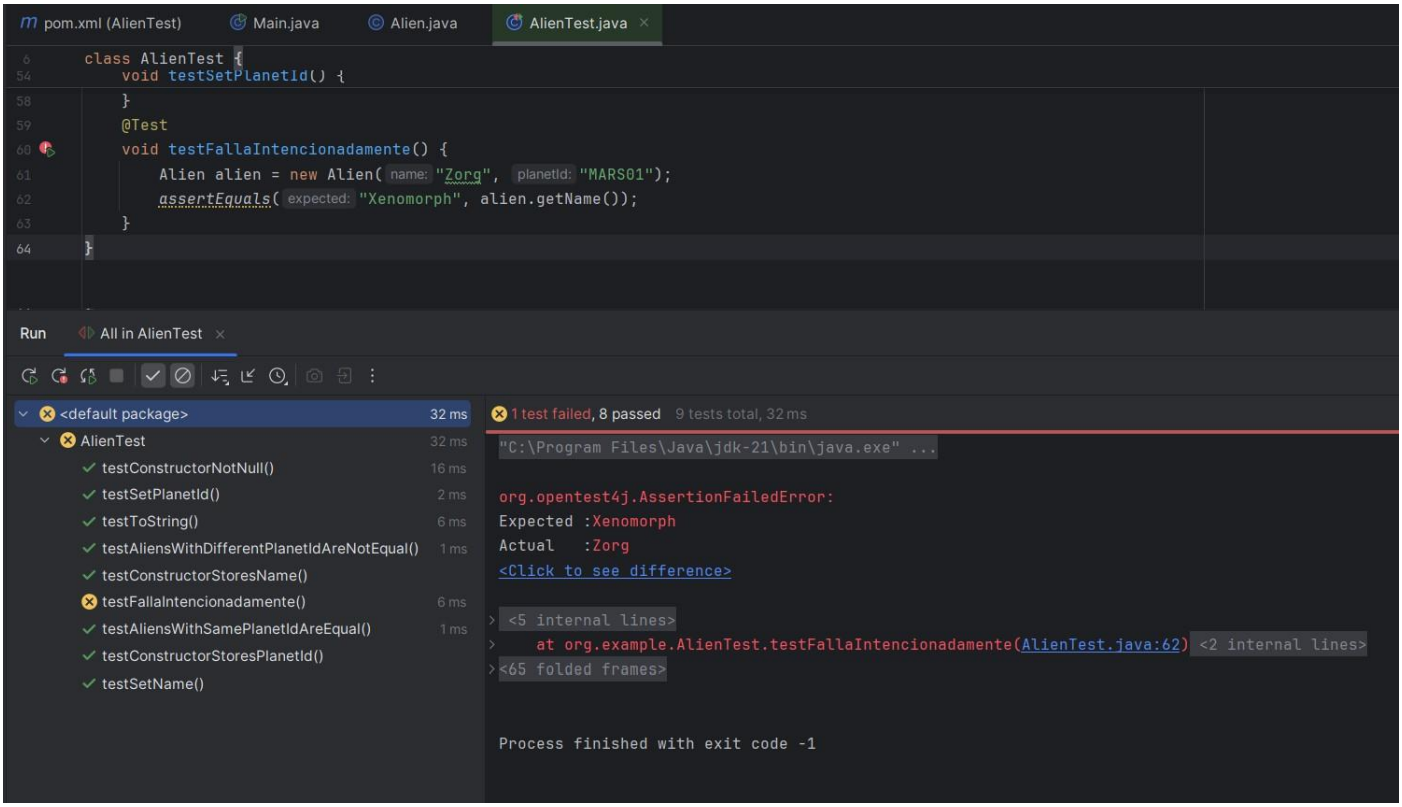

Figura 4 – Test que falla intencionadamente. Se puede observar el mensaje de error de JUnit mostrando el valor expected "Xenomorph" frente al valor actual "Zorg", devuelto por el método getName() de la clase Alien.

pom.xml (AlienTest) | Main.java | Alien.java | AlienTest.java

```
class AlienTest {
    void testSetPlanetId() {
    }
    @Test
    void testFallaIntencionadamente() {
        Alien alien = new Alien( name: "Zorg",  planetId: "MARS01");
        assertEquals( expected: "Zorg", alien.getName());
    }
}
```

Run | All in AlienTest

<default package>	25 ms
AlienTest	25 ms
testConstructorNotNull()	15 ms
testSetPlanetId()	2 ms
testToString()	6 ms
testAliensWithDifferentPlanetIdAreNotEqual()	1 ms
testConstructorStoresName()	
testFallaIntencionadamente()	
testAliensWithSamePlanetIdAreEqual()	1 ms
testConstructorStoresPlanetId()	
testSetName()	

9 tests passed 9 tests total, 25 ms

```
"C:\Program Files\Java\jdk-21\bin\java.exe" ...

Process finished with exit code 0
```

Figura 5 – Corrección del test y ejecución exitosa. Tras corregir el valor *expected*, el entorno confirma que los 9 tests han sido superados con éxito, mostrando el resultado *9 tests passed* en el panel de ejecución de IntelliJ.

pom.xml (AlienTest) | Main.java | Alien.java | AlienTest.java

```
package org.example;

public class Alien {  1 usage

    protected String name;  5 usages
    protected String planetId;  7 usages

    public Alien() {  no usages
        name = "";
        planetId = "";
    }

    public Alien(String name, String planetId) {  no usages
        this.name = name;
        this.planetId = planetId;
    }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

    public String getPlanetId() { return planetId; }

    public void setPlanetId(String planetId) { this.planetId = planetId; }

    @Override
    public boolean equals(Object a) { return this.planetId.equals(((Alien) a).planetId); }

    @Override
    public String toString() { return name + " " + planetId; }
}
```

Coverage | All in AlienTest

Element	Class, %	Method, %	Line, %	Branch, %
org.example	50% (1/2)	77% (7/9)	60% (9/15)	0% (0/2)

Figura 6 – Cobertura de código con Run with Coverage. Se puede observar el panel de cobertura mostrando un 77% en métodos y un 60% en líneas, reflejando las partes de la clase Alien ejecutadas durante los tests."